

10. СЕТЕВЫЕ МОДЕЛИ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ

В современных вычислительных системах особое внимание уделяется организации параллельных процессов. Примером параллельного процесса в работе любого компьютера являются вычисления, проводимые процессором, и одновременный фоновый вывод информации на принтер.

Параллельно работающие системы могут взаимодействовать между собой различным образом, а могут и вообще не взаимодействовать. Параллельные процессы *могут быть ведущими, подчиненными и независимыми*. Подчиненный параллельный процесс управляется (синхронизируется) ведущим. Синхронные процессы могут обмениваться информацией. Независимые процессы являются асинхронными и могут в процессе работы вообще не обмениваться информацией.

Отметим особенности протекания параллельных процессов в вычислительных системах и сетях. Понятно, что только в многопроцессорных системах можно организовать параллельный вычислительный процесс. Вычисления в каждом процессоре могут быть синхронизированы или не синхронизированы между собой. Однако работа процессоров всегда согласуется при обращении к общим ресурсам вычислительной системы или сети.

Использование общих ресурсов вычислительной системы может организовываться по схеме взаимного исключения, когда процесс, первым захвативший общий ресурс, запрещает доступ к нему других параллельных процессов. Эта простейшая схема может быть усовершенствована путем присвоения процессам различных абсолютных или относительных приоритетов.

Синхронизация параллельных процессов может проводиться на аппаратном уровне путем использования общего тактового генератора или на программном (системном) уровне путем обмена сигналами по установленному протоколу. Синхронизация параллельных процессов в вычислительной системе организуется на этапе разработки и включает в себя специальные аппаратные средства и программы операционной системы.

Моделирование параллельных процессов проще всего проводить в рамках специализированных языков моделирования (например, GPSS) и на алгоритмических языках реального времени (например язык АДА). Однако перед написанием программы моделирования необходимо описать алгоритм реализации параллельных процессов, что сделать традиционными методами очень даже не просто.

Одним из распространенных и эффективных способов описания параллельных процессов являются сети Петри. Сети Петри могут быть определены в графической форме в виде специфичного графа или в формально аналитической форме, позволяющей легко перейти к реализации в виде компьютерной программы.

10.1. Определение: Сеть Петри

Сеть Петри **S** включает четыре объекта **{P, T, I, O}**:

- конечное множество *позиций* **P** = { $p_1, p_2, \dots, p_n$ };
- конечное множество *переходов* **T** = { $t_1, t_2, \dots, t_m$ };
- входную функцию **I**: **T** $\leftarrow$ **P**, отображение из позиций в переходы;
- выходную функцию **O**: **T** $\rightarrow$ **P**, отображение из переходов в позиции.

Позиция p_i является *входной позицией* для перехода t_j в том случае, если $p_i \in I(t_j)$; позиция p_i является *выходной позицией* для перехода t_j в том случае, если $p_i \in O(t_j)$.

Пример Примером аналитического описания структуры сети Петри является следующий текст:

Объекты, образующие сеть Петри

$$C = \{P, T, I, O\};$$

Множество позиций

$$P = \{p_1, p_2, p_3, p_4, p_5\}$$

Множество переходов

$$T = \{t_1, t_2, t_3, t_4, t_5\}$$

Входная функция

Выходная функция

$$I(t_1) = \{p_2, p_4\};$$

$$O(t_1) = \{p_1\};$$

$$I(t_2) = \{p_4\};$$

$$O(t_2) = \{p_2\};$$

$$I(t_3) = \{p_1\};$$

$$O(t_3) = \{p_3\};$$

$$I(t_4) = \{p_3, p_5\};$$

$$O(t_4) = \{p_4\};$$

$$I(t_5) = \{p_3\};$$

$$O(t_5) = \{p_5\};$$

Определим *расширенную входную функцию* $I: P \leftarrow T$, отображение из переходов в позиции; и

расширенную выходную функцию $O: P \rightarrow T$, отображение из позиций в переходы.

Эти функции по форме являются обратными к ранее введенным входной и выходной функциям.

Для сети Петри, определенной выше эти функции имеют следующий вид:

2 *Расширенная входная функция*

Расширенная выходная функция

$$I(p_1) = \{t_1\};$$

$$O(p_1) = \{t_3\};$$

$$I(p_2) = \{t_2\};$$

$$O(p_2) = \{t_1\};$$

$$I(p_3) = \{t_3\};$$

$$O(p_3) = \{t_4, t_5\};$$

$$I(p_4) = \{t_4\};$$

$$O(p_4) = \{t_1, t_2\};$$

$$I(p_5) = \{t_5\};$$

$$O(p_5) = \{t_4\};$$

Для наглядного описания сети Петри применяется граф, обладающий двумя типами узлов: *Кружок* является *позицией*, *черта* – *переходом*. Стрелки соединяют *позиции* и *переходы*, при этом стрелка, направленная от позиции к переходу определяет *входную позицию* для этого перехода, а стрелка, направленная от перехода к позиции – *выходную позицию* для этого перехода.

В качестве примера рассмотрим сеть Петри с графом, представленным на рис. 10.1.

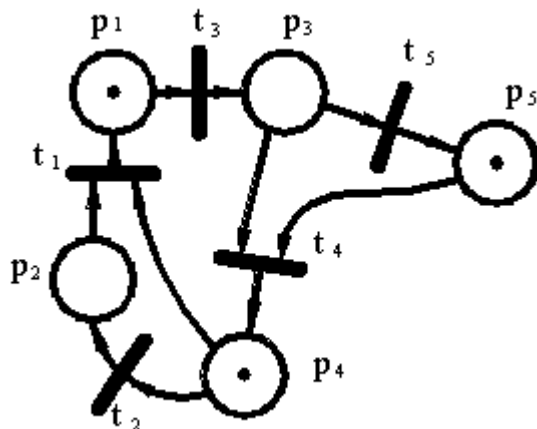


Рис. 10.1. Граф сети Петри. Начальная маркировка сети.

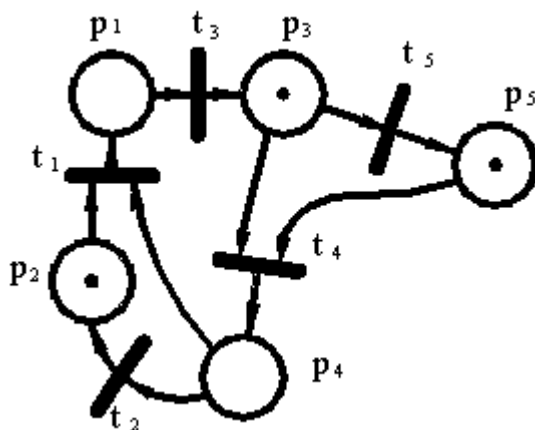


Рис. 10.2. Граф сети Петри. Маркировка после первого такта.

Сеть Петри, представленная в форме графа на рис. 10.1-10.2, соответствует аналитическое описание, приведенное выше в примере.

10.2. Маркировка сети Петри.

Позициям в сети Петри могут быть присвоены *метки* или *фишки*, служащие для определения процесса работы сети. Маркировка сети Петри на рис. 10.1-10.2 представлена в виде точек (меток, фишек), находящихся в некоторых позициях сети.

В аналитической форме маркировка μ сети Петри определяется как отображение множества позиций P в множество неотрицательных целых чисел N

$$\mu: P \rightarrow N.$$

Маркировка может быть задана и в форме вектора $\mu = \{ \mu_1, \mu_2, \dots, \mu_n \}$ ($\mu_i = 0, 1, 2, \dots$), где индекс i при компоненте вектора μ_i совпадает с номером соответствующей позиции p_i . Вообще говоря в каждой позиции может быть ноль, одна или несколько фишек.

Приведем маркировку сети для примера 1, показанную на рис. 10.1.

Маркировка сети $\mu = \{ \mu_1, \mu_2, \mu_3, \mu_4, \mu_5 \}$

$$\mu_1 = 1;$$

$$\mu_2 = 0;$$

$$\mu_3 = 0;$$

$$\mu_4 = 1;$$

$$\mu_5 = 1;$$

В результате аналитическое определение маркированной сети Петри включает пять объектов $\{P, T, I, O, \mu\}$. Вообще говоря число фишек в каждой позиции сети Петри не ограничено. Поэтому каждая сеть Петри может иметь бесконечное число маркировок.

Сеть может быть "оживлена", при этом фишки будут переходить из одной позиции в другую, изменяя маркировку сети. Фишки могут перемещаться по сети только через "активные" (разрешенные) переходы.

Переход активен, если в каждой для него входной позиции имеется хотя бы одна метка.

В примере на рис. 10.1 разрешены (активны) переходы t_2 , t_3 и t_4 , во входных позициях которых p_4 , p_1 и p_5 находятся фишки.

В результате срабатывания перехода число фишек в его входных позициях уменьшается на единицу, а число фишек в его выходных позициях увеличивается на единицу.

Новая маркировка сети, возникшая после срабатывания активных переходов на рис. 10.1, показана на рис. 10.2. В результате фишки оказываются в позициях p_2 , p_3 и p_5 , при этом активным становится переход t_4 .

На следующем такте работы сети произойдет срабатывание перехода t_4 , после чего изменится маркировка сети (рис. 10.3).

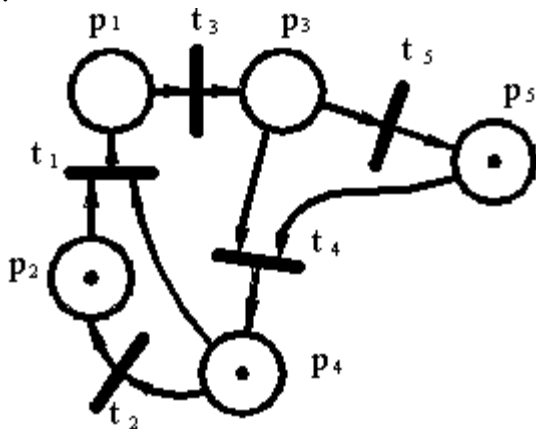


Рис. 10.3. Граф сети Петри. Маркировка после второго такта

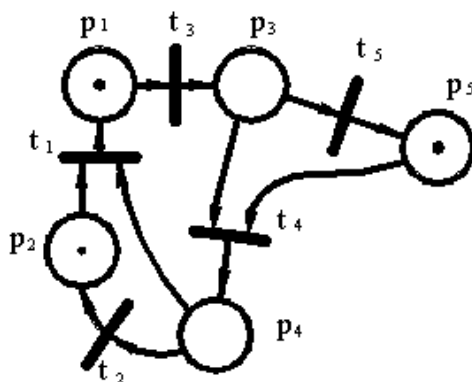


Рис. 10.4. Граф сети Петри. Маркировка после третьего такта

В результате активными становятся переходы t_1 и t_2 , так как в их входных позициях p_2 и p_4 имеется по одной фишке. После срабатывания этих переходов маркировка сети изменится еще раз (рис.10.4).

10.3. Пространство состояний сети Петри

Пространство состояний сети Петри определяется ее маркировкой. Каждое срабатывание активных переходов изменяет маркировку сети и поэтому изменяет ее состояние. Для рассмотренного примера сети Петри (рис. 10.1) маркировке на рис. 10.1 соответствует состояние S_1 , маркировке на рис. 10.2 – состояние S_2 , маркировке на рис. 10.3 – состояние S_3 , маркировке на рис. 10.4 – состояние S_4 и так далее.

Число различных состояний сети Петри, содержащей n позиций и не более N фишек в каждой позиции не превышает число N^n .

При очередном срабатывании активных переходов возникает новая маркировка сети Петри. Поэтому в процессе работы возникают две последовательности:

- последовательность маркировок $\mu^0, \mu^1, \mu^2, \mu^3, \mu^4, \dots$

- и последовательность активных переходов $\{t_{i0}\}, \{t_{i1}\}, \{t_{i2}\}, \{t_{i3}\}, \{t_{i4}\}, \dots$

Эти две последовательности связаны между собой при помощи функции следующего состояния $\delta(\mu^k, t_{ik}) = \mu^{k+1}$, определяющей маркировку сети на следующем такте работы.

Если хотя бы один переход t_{ik} в маркировке μ^k активен, то в результате его срабатывания возникнет новая маркировка μ^{k+1} , непосредственно достижимая из маркировки μ^k .

Понятно, что в результате последовательности переходов из одной маркировки в другую непосредственно достижимую маркировку (из состояния S_i в состояние S_{i+1}) возникает множество достижимости $R(S, \mu^0)$ для сети Петри, определяемое начальной маркировкой μ^0 . Множество достижимости совпадает с наименьшим множеством всех маркировок (всех состояний), достижимых из одного начального состояния сети Петри.

Приведем пример сети Петри, моделирующую работу простейшего компьютера, содержащего устройство ввода, устройство вывода и процессор (рис. 10.5).

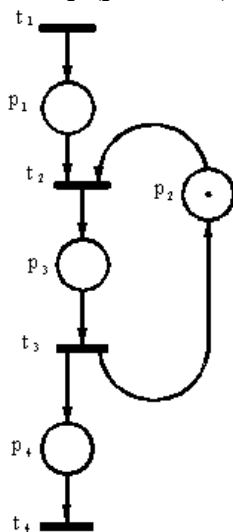


Рис. 10.5. Сеть Петри для простого компьютера:

t_1 – задание помещается во входную очередь,
 p_1 – задание ждет,
 p_2 – процессор свободен,
 t_2 – начало выполнения задания в процессоре,
 p_3 – задание выполняется,
 t_3 – завершение выполнения задания,
 p_4 – задание ждет вывода,
 t_4 – задание выводится.

Поступление очередного задания на вход компьютера моделируется при помощи фишки, поступающей в позицию p_1 через переход t_1 . Если процессор свободен (в позиции p_2 находится фишка), то переход t_2 окажется активным. После его срабатывания фишки в позициях p_1 и p_2 исчезнут и появится фишка в позиции p_3 , означающая обработку задания в процессоре. В результате оказывается активным переход t_3 . После срабатывания этого перехода фишки окажутся в позициях p_2 и p_4 . Из позиции p_4 фишка через активный переход t_4 покинет сеть Петри.

Этот пример показывает, как в сети Петри происходят *события*, т.е. действия, связанные с переходом сети из одного состояния в другое. Для того, что бы *событие* произошло необходимо, что бы выполнилось некоторое *условие* (фишки находятся во всех входных позициях одного из переходов). Эти условия называют *предусловиями для события*. Так, в предыдущем примере для события, состоящего в срабатывании перехода t_2 необходимо выполнить условие, состоящее в наличии фишек в позициях p_1 и p_2 .

10.4. Моделирование параллельных процессов.

В сети Петри два не взаимодействующих события могут происходить независимо друг от друга. Это позволяет моделировать два или несколько асинхронных независимых процессов. Два связанных между собой события должны

происходить одновременно, что также легко моделируется в сети Петри. Считается, что событие происходит мгновенно, за нулевое время, поэтому вероятность одновременного возникновения двух событий принимается равной нулю. Такое событие называется *примитивным*. Примитивные события не могут быть одновременными.

Непримитивными событиями называются события, длительность которых конечна и больше нуля. Такие события могут пересекаться во времени и соответствуют реальным событиям в реальных системах. Ясно, что сеть Петри с примитивными событиями не совсем точно отражает реальные системы. Однако на практике при построении модели на основе сети Петри это обстоятельство легко обходится. Так непримитивное событие заменяется двумя примитивными: "начало непримитивного события", "конец непримитивного события" и условия: "непримитивное событие происходит".

В качестве примера покажем сеть Петри для моделирования работы простого компьютера, в котором время работы процессора учитывается как непримитивное событие.

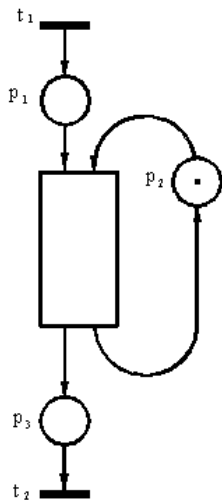


Рис. 10.6. Сеть Петри для моделирования работы простого компьютера. Работа процессора моделируется как непримитивное событие

Применение прямоугольника для моделирования отдельных переходов приводит к значительному упрощению сети Петри, что важно при моделировании сложных систем. Таким образом может быть свернут целый функциональный блок, что эквивалентно применению подпрограмм в программировании.

Обратим внимание на важную особенность сетей Петри. Они в принципе моделируют независимые параллельные процессы как асинхронные. Синхронизация событий возникает автоматически в точках объединения параллельных процессов, в тот момент, когда во всех входных позициях общего перехода возникают фишки.

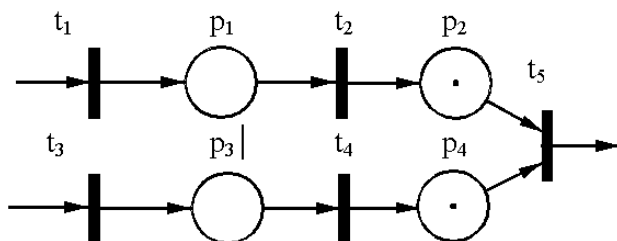


Рис. 10.7. Автоматическая синхронизация двух параллельных процессов при их объединении на переходе t_5

Достаточно просто моделируется и ситуация "захвата ресурсов", когда один из параллельных процессов блокирует работу другого при возникновении некоторого условия.

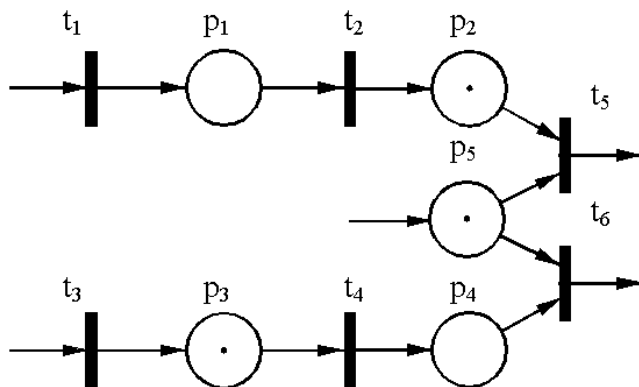


Рис. 10.8 Блокировка одного параллельного процесса другим

На этом рисунке фишка, проходящая по верхнему параллельному процессу, первой достигает позиции p_2 и через разрешенный переход t_5 перемещается дальше. В результате срабатывания перехода t_5 фишка из позиции p_5 удаляется и поэтому нижний параллельный процесс остановится в позиции p_4 . Дальнейшее продвижение фишек через переходы t_5 и t_6 может произойти только после попадания фишки в "разрешающую" позицию p_5 . Заметим, что при наличии фишек в позициях p_2 и p_4 приход фишки в "разрешающую" позицию p_5 синхронизирует работу переходов t_5 и t_6 .

10.5. Моделирование процессора с конвейерной обработкой

Большинство современных процессоров имеют конвейерную организацию вычислительного процесса. Конвейер состоит из ряда операций, которые могут выполняться одновременно. Каждая операция конвейера получает исходные данные от предыдущей операции и передает результат последующей.

Наибольший выигрыш времени вычислений в процессоре с конвейером достигается при выполнении однородных операций, например, при сложении ряда чисел. Каждая

операция сложения чисел с плавающей запятой может быть представлена как следующая последовательность элементарных операций:

1. сравнить экспоненциальные части чисел и найти разность между ними,
2. если необходимо, переместить точку в мантиссе меньшего числа,
3. сложить мантиссы,
4. нормализовать сумму,
5. вычислить новое значение экспоненты,
6. проверить это значение на переполнение,
7. сформировать формат суммы.

Каждая операция может быть выполнена специальным аппаратным блоком. После выполнения первой операции результат передается во второй блок, при этом первый блок оказывается свободным. Поэтому в нем одновременно с работой второго блока по сложению первых двух чисел может быть выполнена первая операция по сложению третьего и четвертого числа. Если последовательность слагаемых достаточно велика, то в конвейере может одновременно складываться семь чисел, при этом результаты сложения выдаются в темпе один такт вместо семи тактов при обычной организации вычислений.

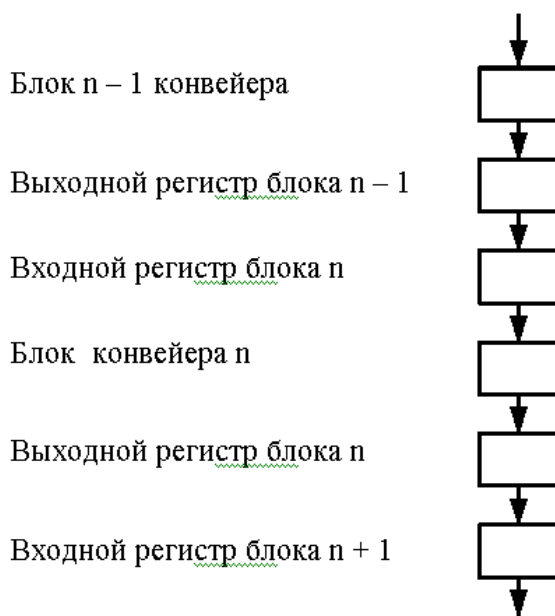


Рис. 10.9. Организация конвейерной обработки чисел в процессоре

Синхронизация работы блоков конвейера выполняется различным образом. При синхронной организации информация из блока в блок передается в темпе работы самого медленного блока. При более сложной асинхронной организации информация после обработки в блоке n поступает в его выходной регистр и ждет там до тех пор, пока не освободится входной регистр следующего блока $n + 1$. В асинхронном режиме скорость вычислений в среднем выше, хотя организация вычислений более сложна.

Таким образом при асинхронной конвейерной обработке для каждого вычислительного блока n возникают следующие состояния сети Петри:

- входной регистр блока n заполнен,
- входной регистр блока n свободен,
- выходной регистр блока n заполнен,
- выходной регистр блока n пуст,

- вычислительный блок n занят,
- вычислительный блок n свободен.
-

Построим модель асинхронной конвейерной обработки в форме сети Петри.

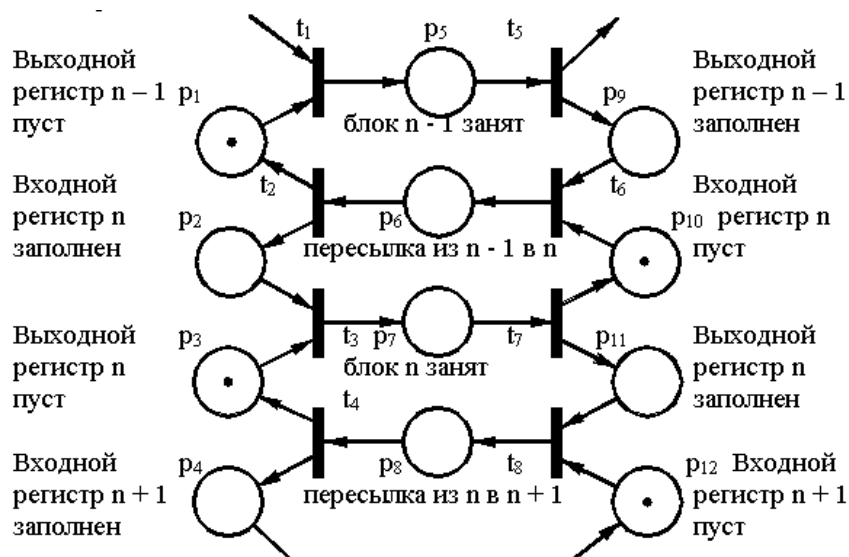


Рис. 10.10. Модель асинхронной конвейерной обработки информации в процессоре

Устройство управления процессом конвейерной обработки в процессоре (рис. 10.10) находится в исходном состоянии и готово к работе. При поступлении второй фишки в переход t_1 он срабатывает и устанавливает фишку в позицию p_5 . После обработки информации в этом блоке фишка переходит в позицию p_9 , обозначая окончание работы блока $n-1$ конвейера. В результате оказывается разрешенным переход t_6 и после его срабатывания фишка оказывается в позиции p_6 , обозначая передачу информации во входной регистр следующего блока n . После этого блок n начинает обработку

информации, а блок $n - 1$ освобождается и оказывается готовым к работе.

10.6. Кратные функциональные блоки компьютера

Для повышения производительности компьютера в состав процессора часто вводятся дополнительные регистры и другие функциональные блоки (блоки сложения и умножения чисел, логические блоки), позволяющие выполнять несколько инструкций программы одновременно. Организация параллельных вычислений на основе кратных функциональных блоков должна быть такой, что бы результат работы по линейной программе и с использованием кратных блоков был бы одинаков.

Пусть в программе требуется последовательно выполнить две операции a и b . Если для выполнения операции b не требуются данные, полученные при выполнении операции a , то операцию b можно выполнить одновременно или раньше операции a .

Управление программами такого типа основано на использовании *таблиц резервирования*. В такую таблицу заносятся блоки и регистры, необходимые для выполнения данной операции. Если эти блоки свободны, то такую операцию можно выполнять. Процессор резервирует эти блоки и выдает команду на выполнение этой операции. Если хотя бы один необходимый блок занят, то такая операция не выполняется и процессор ждет, когда возникнут условия, необходимые для выполнения этой операции.

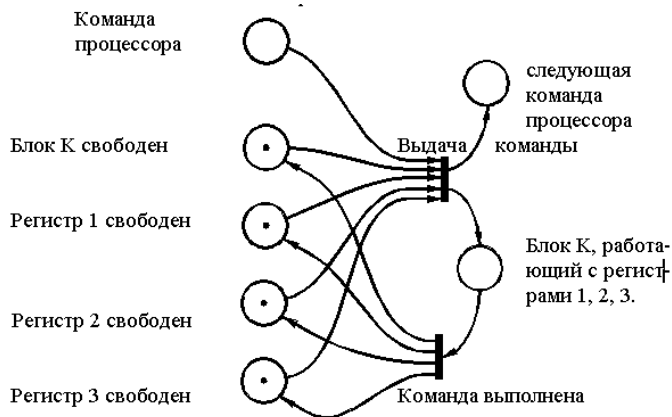


Рис. 10.13. Часть сети Петри, моделирующей устройство управления процессора с кратными функциональными блоками

Такая ситуация может быть промоделирована сетью Петри (рис. 10.13). Поставим в соответствие каждому функциональному блоку и регистру свою позицию. Если регистр свободен, то пометим его фишкой. В кратные регистры поместим соответствующее число фишек.

Перед тем, как выдать очередную команду процессор проверяет, что необходимые для ее выполнения регистры свободны (в них находятся фишки). Если это так, то команда начинает выполняться и подается команда к проверке готовности системы к выполнению следующей команды.

Приведенная схема очень проста и показывает только идеологию параллелизма, основанного на введении кратных функциональных блоков. Понятно, что некоторые функциональные блоки могут использовать общие регистры, а содержимое регистров может управлять порядком выполнения машинных команд.

10.7. Сети Петри и программирование

Сеть Петри является удобным аппаратом для представления работы условных операторов компьютерной программы и позволяет проанализировать порядок выполнения операторов в программе. Обычным способом решения этой задачи является составление блок – схем.

Рассмотрим применение сети Петри на конкретном примере. Запишем простейшую программу, содержащую условные операторы (Паскаль)

```
begin
    input (y1);
    input (y2);
    y3 := 1;
    while y1 > 0
        do begin
            if odd (y1)  \нечетность числа
            then begin
                y3 := y1 + y2;
                y1 := y1 - 1;
            end;
            y2 := y2*y2;
            y1 := y1 - 2;
        end;
        output (y3);
    end;
```

Операторы в этой программе можно разделить на *условные операторы*, определяющие логику работы программы, и *все остальные*. На блок – схеме программы принято условный оператор принято обозначать ромбом, а все остальные – прямоугольником. Нарисуем блок – схему, соответствующую этой программе.

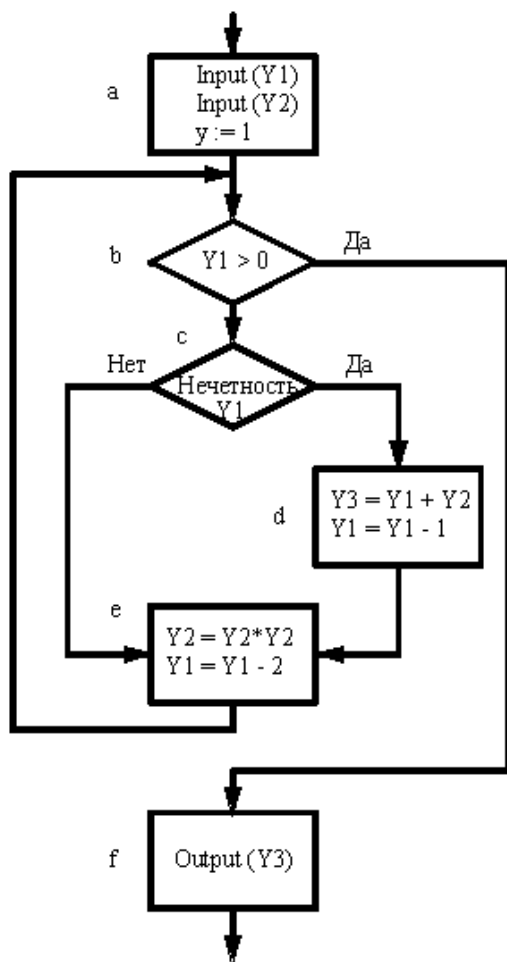


Рис. 10.12. Блок – схема компьютерной программы.

Отметим разницу между блок – схемой и сетью Петри. В блок – схеме действия происходят в узлах (прямоугольниках или ромбах), а в сети Петри действия происходят в переходах. Поэтому для перевода блок – схемы программы в сеть Петри необходимо прямоугольники и ромбы заменить переходами, а на соединяющих их дугах поместить позиции. Правила перевода иллюстрируются на рис. 10.13.

Отметим, что переходам, соответствующим условным операторам, предшествует позиция, содержащая два выхода, один из которых соответствует переходу Да, а второй – Нет.

На рис. 10.14.представлена сеть Петри, соответствующая блок – схеме на рис. 10.12. Для запуска сети Петри необходимо поместить фишку во входную позицию и анализировать ее движение по сети.

Сравнивая эти схемы, составленные для простейшей программы, можно утверждать, что сеть Петри не дает каких либо важных преимуществ при анализе управляющей структуры компьютерной программы. Однако ситуация меняется, если необходимо разработать программу, реализующую процесс параллельных вычислений с использованием общего ресурса.

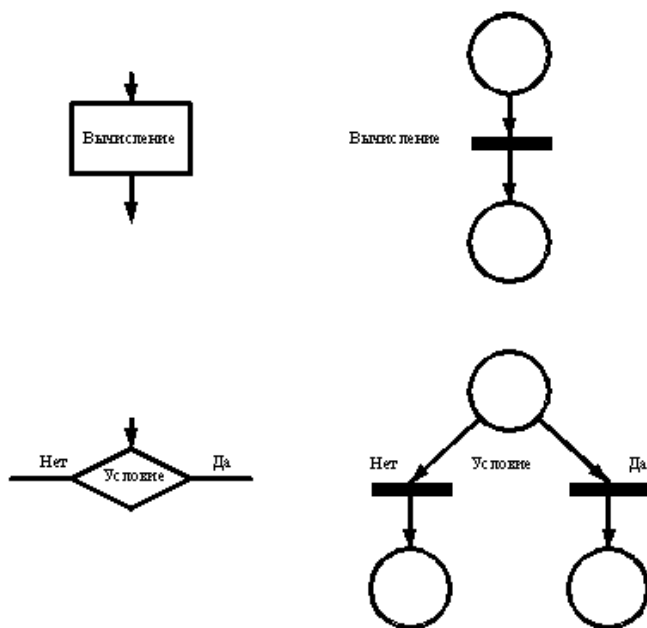


Рис. 10.13. Правила перевода блок – схемы компьютерной программы в сеть Петри.

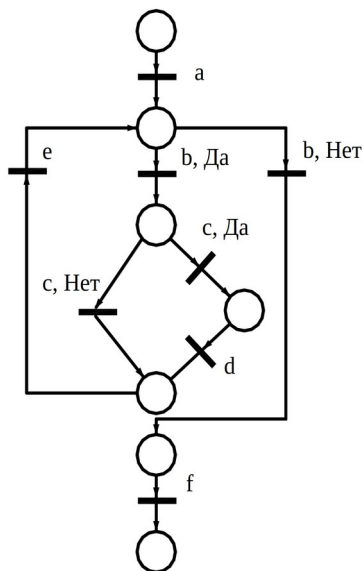


Рис. 10. 14. Сеть Петри, соответствующая блок – схеме программы, представленной на рис. 10.12

10.8. Взаимно исключающие параллельные процессы

Очень часто два параллельных вычислительных процесса используют общий аппаратный ресурс, например, общую часть оперативной памяти. Для правильной работы вычислительной системы необходимо обеспечить поочередный доступ к общему ресурсу и блокировать доступ к занятому ресурсу. Эта ситуация изображена на следующем рисунке.

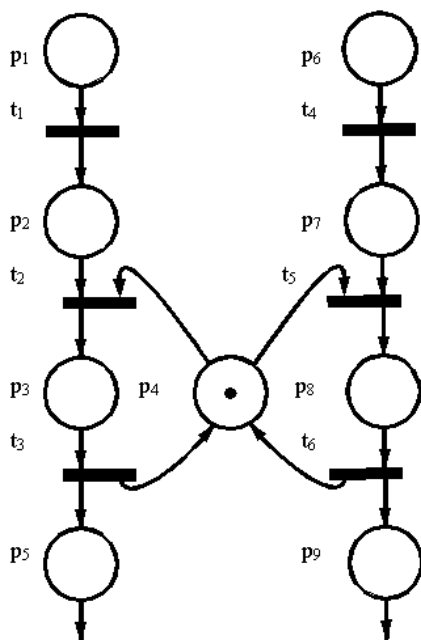


Рис. 10.15. Сеть Петри для взаимно исключающих процессов

На этой схеме позиции p_3 и p_8 соответствуют критическим сегментам программы, в которых они обращаются к общему аппаратному ресурсу. Этот ресурс захватывает тот процесс, в котором раньше появится фишка в позиции p_2 или p_7 . Пусть появилась фишка в позиции p_2 . Тогда переход t_2 оказывается активным и фишки из позиции p_2 и "разрешающей" позиции p_4 поступают в позицию p_3 , соответствующую общему аппаратному ресурсу для двух процессов. В результате переход t_3 оказывается активным и после его срабатывания фишки поступают в позицию p_5 и "разрешающую" позицию p_4 . Все время, пока первый процесс находится в позиции p_3 , критическая позиция p_8 оказывается заблокированной для второго процесса.

Замечание. В рассматриваемых сетях Петри рассматриваются только примитивные, мгновенно

происходящие события, и соответствующие им примитивные переходы. Поэтому вероятность одновременных событий принимается равной нулю.

Замечание. Сети Петри моделируют только логику работы программы или технического устройства.

Моделирование систем с флагами. Одним из наиболее распространенных механизмов синхронизации является использование флагов (семафоров), указывающих на возможность использования в программе соответствующего блока. Флаг – это элемент данных, принимающих два значения:

"0" – блок свободен,

"1" – блок занят.

Такие системы очень просто моделируются сетями Петри.

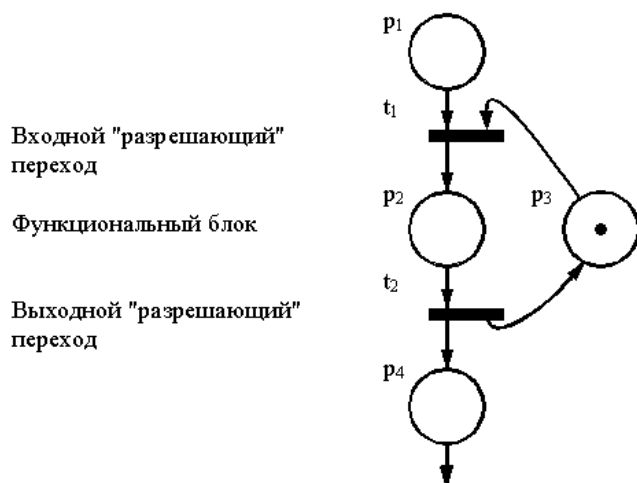


Рис. 10.16. Моделирование системы с флагом

При поступлении фишки в позицию p_1 переход t_1 оказывается разрешенным. После срабатывания этого перехода фишка попадает в позицию p_2 , при этом переход t_1 запрещен (нет фишки в позиции p_3).

Переход t_2 разрешен и после его срабатывания фишки попадают в позиции p_3 и p_4 . Таким образом в позиции p_2

может находиться только одна фишка (только одна заявка находится в функциональном блоке, моделируемым позицией p_2).

По этой схеме действует пара блоков SEIZE и RELEASE языка GPSS.

10.9. Анализ сетей Петри

Построение модели вычислительной системы в форме сети Петри является только предварительным этапом моделирования. Для изучения свойств моделируемой системы необходимо провести анализ сети. На практике для этого необходимо как правило написать компьютерную программу и задавая различные исходные данные изучить работу моделируемой системы. Широкие возможности для моделирования открывает система Simulink пакета Matlab. Определим основные свойства сетей Петри, которые являются основными задачами моделирования.

Безопасность. Позиция сети Петри называется безопасной, если в ней может быть не более одной фишки. Сеть Петри называется безопасной, если все ее позиции безопасны.

Понятно, что безопасная сеть Петри позволяет упростить техническую реализацию моделируемой системы. Такая система реализуется в обычной бинарной логике на базе обычной элементной базы (триггеров, логических микросхем). Безопасные позиции соответствуют логическому условию (есть фишка – условие истинно, нет фишки – условие ложно). Если в позиции может быть несколько фишек, то такая логическая интерпретация невозможна. Таким образом только в безопасной сети возможно простое моделирование событий и условий.

Сеть Петри, являющаяся безопасной в начальный момент времени, может быть сделана принудительно безопасной (кратных дуг в сети быть не должно). Рассмотрим способ обеспечения безопасности на примере. Структура сети Петри не гарантирует безопасность, так как при поступлении в

начальную позицию нескольких фишек они могут продвигаться по автоматически разрешенным переходам или накапливаться в некоторых позициях.

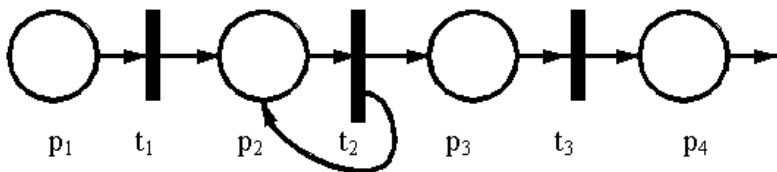


Рис. 10.17. Пример небезопасной сети Петри

Переход t_2 на этом рисунке размножает фишки, которые накапливаются в позиции p_2 .

Безопасная сеть может быть создана из небезопасной различными способами. Например, можно ввести дополнительные условия, состоящие в принудительном запрещении попадания в любую позицию более одной фишки.

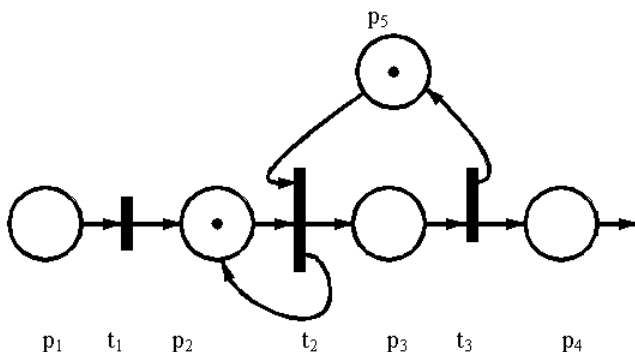


Рис. 10.18. Сеть Петри, которая начиная с позиции p_3 становится безопасной.

Для принудительного обеспечения безопасности, например, позиции p_3 достаточно ввести дополнительную позицию p_5 , реализующую условие: *в позиции p_3 нет фишки* (рис. 10.18).

Понятие безопасности сети Петри естественным образом расширяется на понятие *ограниченности*, состоящее в том, что в позиции сети не может быть больше чем m фишек.

Безопасная позиция может быть аппаратно реализована триггером, а ограниченная – счетчиком. Неограниченную сеть Петри реализовать нельзя.

Сохраняемость. Для некоторых вариантов сетевых моделей важным является понятие сохраняемость. Сеть Петри называется сохраняющей, если в процессе работы сохраняется число фишек, перемещающихся в сети. Это важно, когда фишкам соответствуют какие либо ресурсы, например, блоки памяти или устройства ввода и вывода информации.

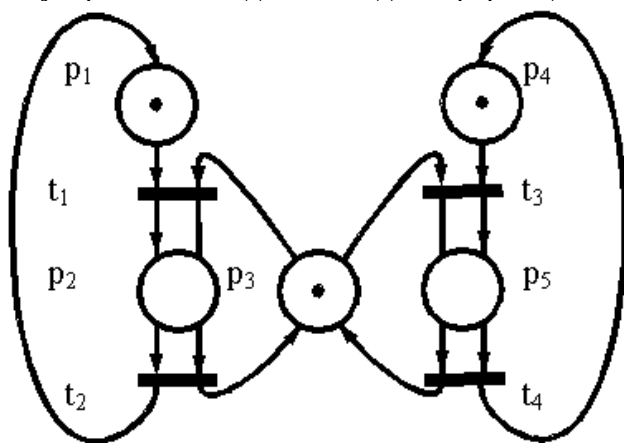


Рис. 10.19. Пример сохраняющей сети Петри.

В процессе функционирования модели эти блоки не могут возникать или уничтожаться.

Условие сохраняемости является очень жестким. В качестве примера рассмотрим сеть, моделирующую взаимно исключающие процессы (рис. 10.19) (сравни с рис. 10.15). Если в позицию p_1 фишка попадет раньше, чем в позицию p_4 , то сработает переход t_1 и в позицию p_2 попадут две фишки (фишки из позиций p_1 и p_3 будут удалены). Затем через переход t_2 фишки вернутся в позиции p_1 и p_3 . Ситуация, когда в позиции p_1 и p_4 фишки попадут одновременно невозможна, так как переходы примитивны.

Не сохраняющая сеть Петри показана на рис. 10.20.

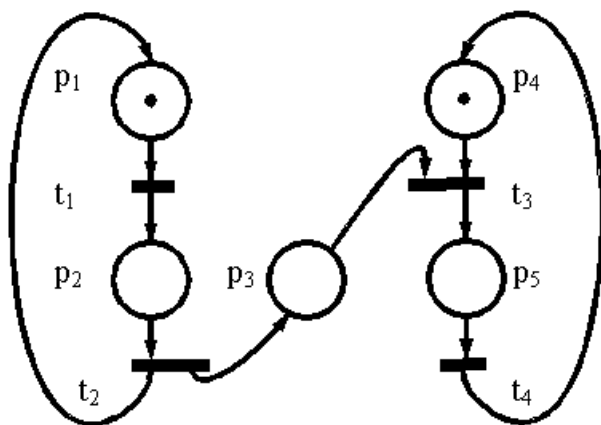


Рис. 10.20. Не сохраняющая сеть Петри

Активность. В процессе функционирования сети Петри могут возникать ситуации, когда некоторые фишки попадают в тупики, в результате чего некоторые переходы сети перестают быть активными. Возможно ситуация, когда неактивны все переходы сети, в результате чего движение фишек в ней прекращается. Понятно, что это ситуация зависит от начального расположения меток в сети (от начальной маркировки).

Достижимость. С понятием активность близко связано понятие достижимость. Под достижимостью маркировки μ_j из начальной (из другой) маркировки μ_k понимается существование последовательности маркировок, приводящих сеть из состояния с маркировкой μ_k в состояние с маркировкой μ_j .

10.10. Дерево достижимости сети Петри

Основным методом анализа сетей Петри является построение дерева достижимости, позволяющий найти множество маркировок, достижимых из заданной начальной маркировки сети Петри.

Построение дерева достижимости легко выполняется путем счета по компьютерной программе, написанной для данного варианта сети.

Рассмотрим процесс построения дерева достижимости на конкретном примере.

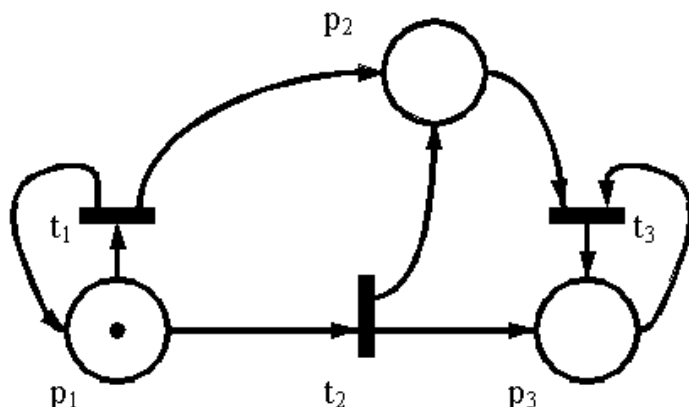
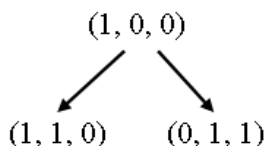


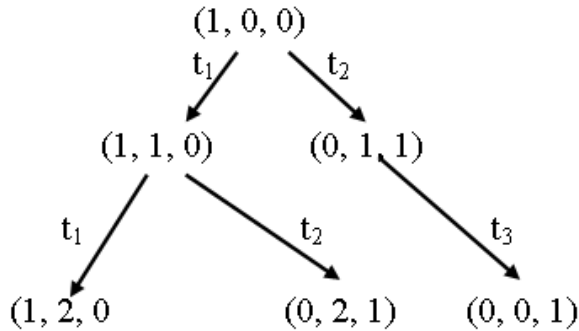
Рис. 10.21. Сеть Петри, для которой строится дерево достижимости.

Начальная маркировка сети $\mu_0 = (1, 0, 0)$ – фишка находится в позиции p_1 . В этот момент разрешены два перехода: t_1 и t_2 . Дерево достижимости должно охватывать все возможные варианты эволюции сети. В результате срабатывания перехода t_1 будет достигнута маркировка $\mu_1 = (1, 1, 0)$, а в результате срабатывания перехода t_2 – маркировка $\mu_2 = (0, 1, 1)$. Фрагмент дерева достижимости для первого шага имеет следующий вид:



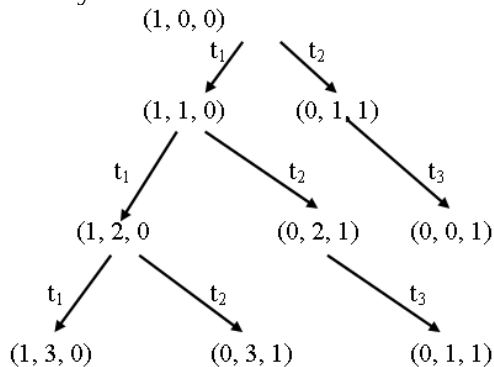
Из маркировки $\mu_1 = (1, 1, 0)$ можно запустить переход t_1 и получить маркировку $\mu_3 = (1, 2, 0)$ и запустить переход t_2 и получить переход $\mu_4 = (0, 2, 1)$. Из маркировки $\mu_2 = (0, 1, 1)$

можно запустить переход и получить маркировку $\mu_5 = (0, 0, 1)$.
В результате получим



дерево достижимости для второго шага построения.

Для третьего шага получим

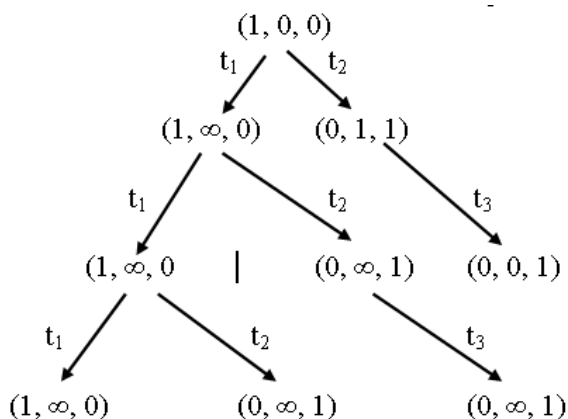


Продолжая таким образом заметим, что в результате для конечной сети Петри строится бесконечное дерево достижимости. Приведение такого дерева к конечному виду проводится следующим способом. Введение новых маркировок ограничивается на каждом шаге граничными вершинами. Это:

- *пассивные маркировки*, из которых никакие переходы невозможны (такой является маркировка $\mu_5 = (0, 0, 1)$, полученная в результате срабатывания переходов t_2 t_3 , и

- ранее встречавшиеся маркировки, из которых повторяются уже построенные фрагменты дерева достижимости. Такой является маркировка $\mu_2 = (0, 1, 1)$, полученная в результате срабатывания последовательности переходов $t_1 t_2 t_3$.

Для этого дерева только маркировка $\mu_5 = (0, 0, 1)$ является пассивной. Для всех остальных маркировок некоторые переходы сети оказываются разрешенными, что приводит к постепенному накоплению фишек в позиции p_2 . Это означает, что сеть имеет бесконечное множество маркировок, что отмечено на конечном дереве достижимости для этой сети.



Если последовательность маркировок ограничена (символ ∞ отсутствует в дереве достижимости), то число состояний сети Петри конечно. Дерево достижимости является графом состояний для этой сети. В этом практически важном случае находить множество всех достижимых маркировок (множество состояний сети) можно простым перебором. Понятно, что только ограниченная сеть Петри является безопасной, в которой число фишек в позициях не может превышать некоторое конечное число. Такая сеть аппаратно осуществима. Пример ограниченной сети Петри приведен на следующем рисунке.

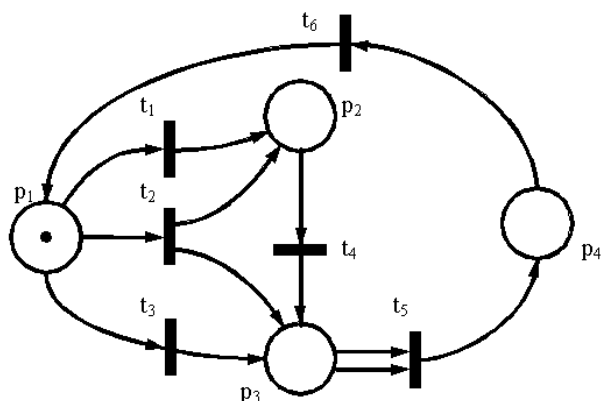
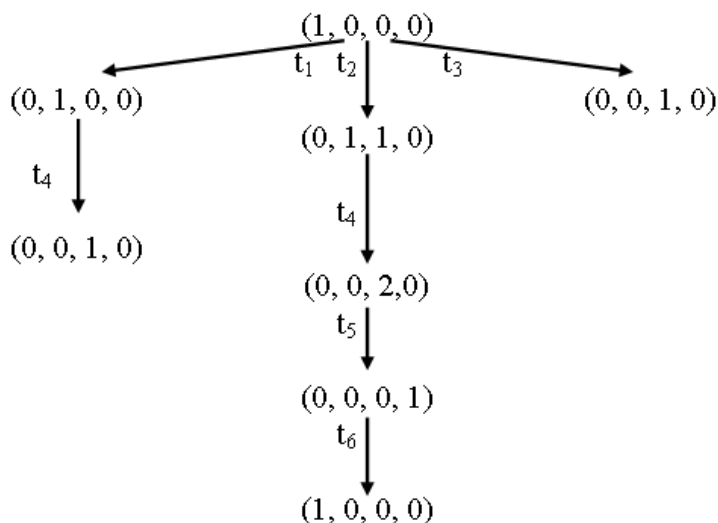


Рис. 10.22. Ограниченная сеть Петри

Дерево достижимости для этой сети имеет следующий вид:



Граф достижимости показывает, что сеть Петри ограничена и безопасна. Дерево достижимости позволяет легко проверить, является ли сеть сохраняющей или нет. Для этого достаточно сложить количество фишек для каждой маркировки (для каждой вершины дерева достижимости) и

сравнить суммы. Если они одинаковы, то сеть сохраняющая. Сеть на рис. 10.22 не является сохраняющей.

Анализ дерева достижимости для ограниченной сети Петри позволяет определить все его свойства, перечисленные выше. Если сеть неограниченна (в разметке дерева достижимости присутствует символ ∞), то анализ сети затруднен. Наличие символа ∞ означает потерю части информации о свойствах сети.

Основным недостатком сетей Петри являются трудности моделирования времени работы (времени задержки) переходов. Поэтому классические сети Петри позволяют эффективно моделировать логику работы программ и технических устройств, но имитационное моделирование временных характеристик в них затруднено. Поэтому разработаны различные варианты расширения возможностей сетей Петри, приспособленных к решению различных конкретных задач.

10.11. Е – сеть

Наиболее современным и продвинутым к задачам графического программирования и моделирования вычислительных систем являются Е–сети (от английского evaluation – вычисления).

Позиции в Е-сети.

Е-сети включают несколько типов позиций:

- позиции (простые),
- очереди,
- разрешающие позиции.

Меткам (фишкам) в Е-сети могут присваиваться атрибуты.

Переходам приписывается время задержки и функции преобразования атрибутов меток.

В позицию может входить и выходить только одна дуга

Переходы в Е-сети.

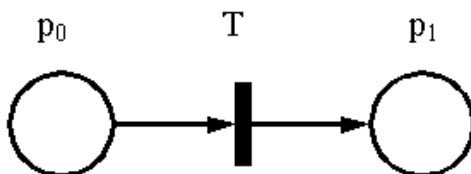
Любой переход в Е-сети описывается следующим выражением

$$t_i = (S, \tau_i, \rho_i),$$

где τ_i – время задержки в переходе t_i , ρ_i – функция преобразования атрибутов метки (фишки, транзакта) в переходе t_i , S – тип перехода.

Перечислим основные типы переходов в Е-сети.

"Простой" переход T . Его изображение аналогично изображению перехода в сети Петри.



Для срабатывания перехода необходимо, чтобы во входной позиции p_0 находилась метка, а в выходной позиции p_1 отсутствовала.

Таблица истинности для этого перехода имеет следующий вид

Позици и	Результат срабатывания перехода T	
	До срабатыван ия	После срабатывания
p_0	1	0
p_1	0	1

Наличие метки в позиции обозначено 1, отсутствие - 0.

T -переход t_i может моделировать процесс обслуживания транзакта (время задержки τ_i) с изменением атрибутов, присвоенных транзакту (функция преобразования ρ_i).

Разветвлению в сетевой модели соответствует F переход. Его графическое изображение имеет следующий вид.

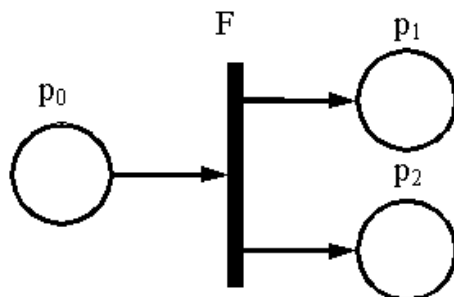


Таблица истинности для F перехода имеет следующий вид.

Позици и	Результат срабатывания перехода F	
	До срабатыван ия	После срабатывания
p ₀	1	0
p ₁	0	1
p ₂	0	1

F переход соответствует разветвлению потока транзактов в сетевой модели.

J переход соответствует объединению потоков транзактов или выполнению условия (наличие меток в двух входных позициях). Его графическое изображение имеет следующий вид.

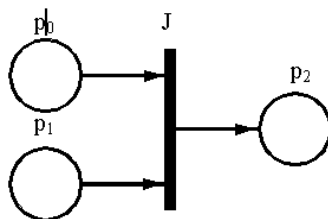


Таблица истинности для J перехода имеет следующий вид.

Позиции	Результат срабатывания перехода J	
	До срабатывания	После срабатывания
p_0	1	0
p_1	1	0
p_2	0	1

Переход срабатывает при наличии меток в позициях p_0 и p_1 .

X переход моделирует в сети переключение потоков транзактов в соответствии с дополнительным условием, помещенным в специальную позицию, обозначенную квадратом.

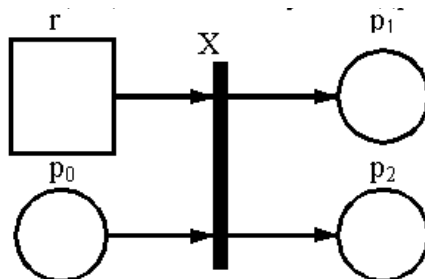


Таблица истинности для X перехода имеет следующий вид.

Позиции	Результат срабатывания перехода X	
	До срабатывания	После срабатывания
r	0	0
p_0	1	0
p_1	0	1

p_2	0	0
-------	---	---

Таблица истинности для X перехода (продолжение).

<i>Позиции</i>	Результат срабатывания перехода X	
	До срабатывания	После срабатывания
r	0	0
p ₀	1	0
p ₁	0	1
p ₂	1	1
r	1	0
p ₀	1	0
p ₁	0	0
p ₂	0	1
r	1	0
p ₀	1	0
p ₁	1	1
p ₂	0	1

Разветвление потока информации производится в соответствии с состоянием разрешающей позиции r (наличия или отсутствия метки в этой позиции), что соответствует обычному условному оператору.

Y переход соответствует условному выбору направления для потока транзактов. Его графическое представление имеет следующий вид.

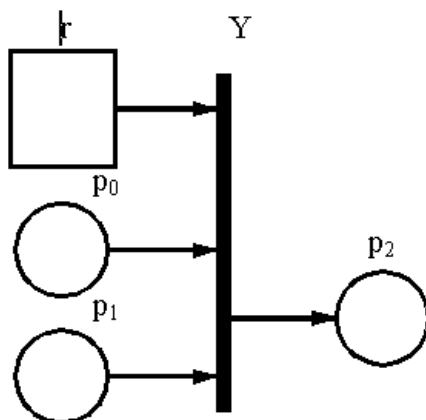


Таблица истинности для Y перехода имеет следующий вид.

<i>Позиции</i>	Результат срабатывания перехода Y	
	До срабатывания	После срабатывания
r	0	0
p_0	1	0
p_1	1	1
p_2	0	1
r	0	0
p_0	1	0
p_1	0	0
p_2	0	1
r	0	0
p_0	0	0
p_1	1	0
p_2	0	1
r	1	0

p_0	1	1
p_1	1	0
p_2	0	1
r	1	0
p_0	1	0
p_1	0	0
p_2	0	1
r	1	0
p_0	0	0
p_1	1	0
p_2	0	1

При помощи Y перехода можно реализовать приоритетность одного из двух потоков транзактов в сетевой модели.

Е-сеть легко позволяет реализовать стандартные функциональные узлы, применяемые при имитационном моделировании вычислительных систем. Приведем примеры.

Генератор транзактов строится на стандартных элементах Е - сети и имеет следующую схему.

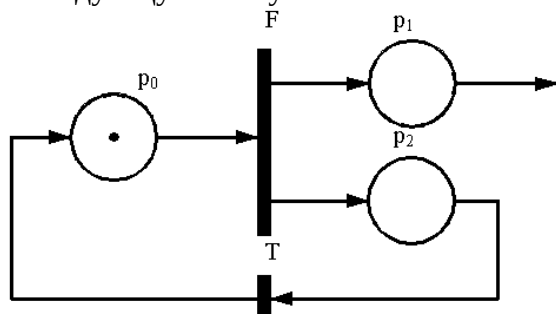


Рис. 10.23. Генератор транзактов на элементах Е - сети

Генератор транзактов использует два перехода F и T . Метка, находящаяся в позиции p_0 запускает переход F и попадает в позиции p_1 и p_2 . Из позиции p_1 метка переходит в сетевую модель, генерируя очередной транзакт. Из позиции p_2 метка через переход T возвращается в исходную позицию p_0 .

Терминатор (аккумулятор) уничтожает транзакты, прошедшие сетевую модель. Его схема имеет следующий вид.

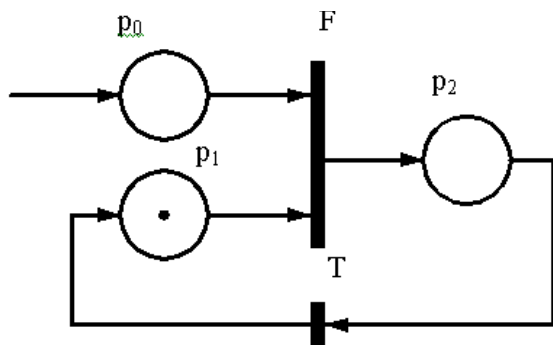


Рис. 10.24. Терминатор (аккумулятор) транзактов

Очередь. Простейшая схема очереди строится на T переходах по следующей схеме.

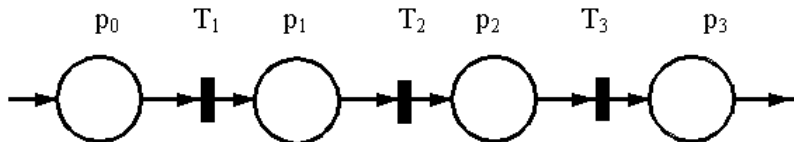


Рис. 10.25. Простейшая очередь на элементах Е-сети

Транзакт (метка) становится в очередь в позиции p_0 и последовательно проходит по ней, покидая очередь в позиции p_3 .

Канал обслуживания начинает обработку очередного транзакта, если он свободен, и задерживает его на время обслуживания. Его схема имеет следующий вид.

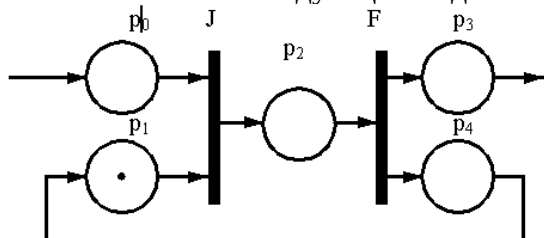
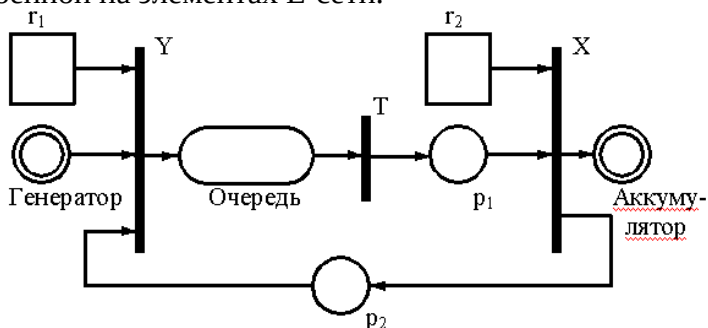


Рис. 10.26. Канал обслуживания на элементах Е-сети

Транзакт из сетевой модели (метка) поступает в позицию p_0 . Если канал свободен (в позиции p_1 есть метка), то транзакт из p_0 проходит в позицию p_2 , моделируя обслуживание. Время обслуживания задается в переходах J или F. После окончания обслуживания транзакт (метка) поступает в позицию p_3 и позицию p_4 . Из позиции p_3 транзакт уходит в сетевую модель, а из позиции p_4 возвращается в позицию p_0 , пропуская очередной транзакт в канал обслуживания.

Обратим внимание на то, что Е-сеть является безопасной, так как в каждой позиции в ней может быть не больше одной метки.

Приведем пример имитационной модели компьютера, построенной на элементах Е-сети.



10.27. Модель компьютера на элементах Е-сети

Модель компьютера включает в себя блок "Генератор", схема которого приведена на рис. 10.23, "Аккумулятор" (схема на рис.10.24) и "Очередь" (схема на рис. 10.25). Обработка заданий в процессоре имитируется задержкой задания в переходе Т на фиксированное время.

Первое задание из Генератора через переход Y проходит в Очередь и из нее попадает в процессор T . Если процессор успел выполнить это задание, то переход X по условию g_2 отправляет его в Аккумулятор. Если выполнение задания не закончено, то оно по условию g_2 отправляется в

позицию p_2 , а из нее вновь поступает в Очередь для дальнейшего выполнения в процессоре. Условие g_1 определяет приоритет, в соответствии с которым в очередь направляются задания или из Генератора, или незавершенные задания из p_2 .

Отметим удобство создания сетевых моделирования в рамках идеологии Е-сетей по сравнению с сетями Петри. Очевидно, что все свойства и методы анализа сетей Петри применимы и к Е-сетям.

Дополнительным преимуществом Е-сетей является их согласованность с возможностями программы имитационного моделирования Simulink пакета Matlab.